

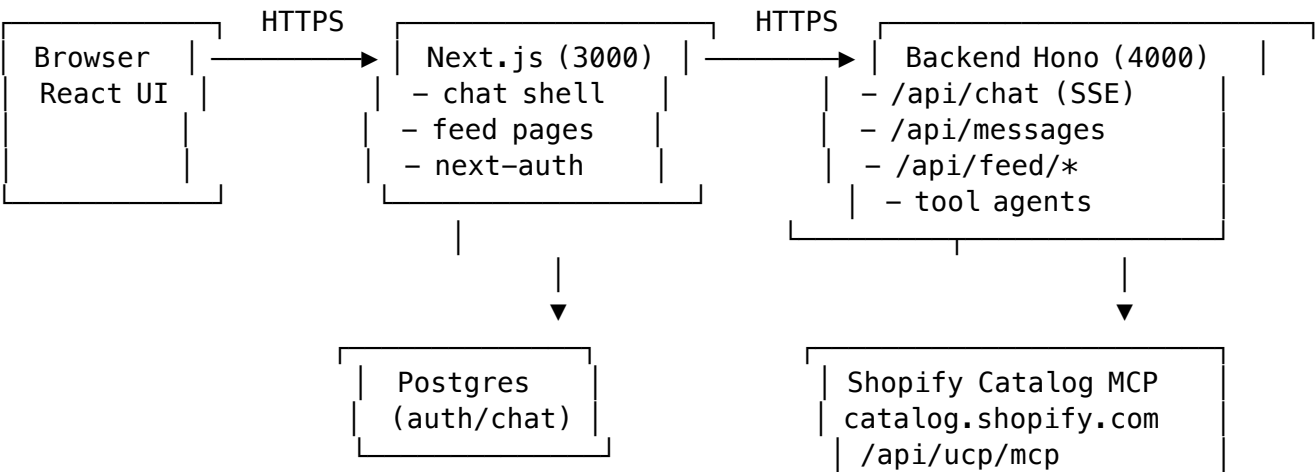
Contents

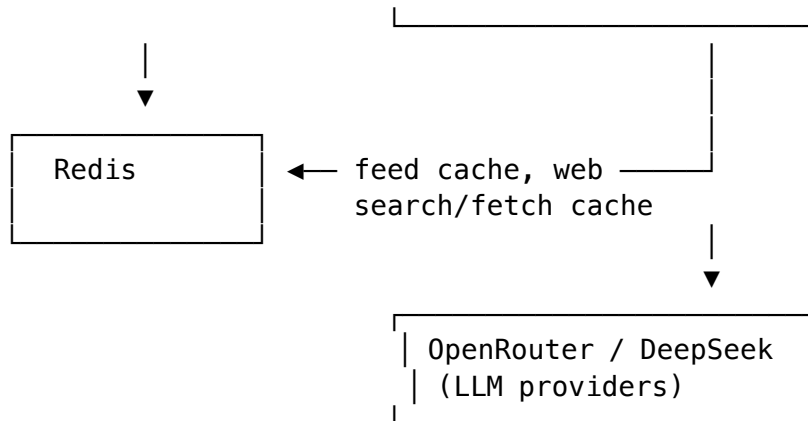
Technical Document	1
Architecture	1
Why three processes	2
Process layout	2
Frontend (frontend/)	2
Backend (backend/src/)	2
Storage	3
Agent design	3
Registry	3
Track 1 tools (backend/src/lib/agents/track1-shopping/tools/)	3
Search memo	4
Per-chat variation	4
Stop conditions	4
Generative UI contract	4
Defensive parsing of assistant text	5
Streaming and persistence	5
Stream	5
Persistence	5
Auto-resume	6
Caching strategy	6
Failure handling	6
Security and secrets	7
Limitations	7
How to extend	8

Technical Document

Architecture

Three-process system. Frontend (Next.js 16 App Router) talks to a Hono/Node backend (tsx src/server.ts) which is the only process that talks to the Shopify Catalog MCP server. Postgres + Redis run locally in Docker.





Why three processes

- Frontend owns auth, message persistence reads, generative UI rendering, and the recommendation feed pages. It does not hold any Shopify or LLM credentials — it always proxies through the backend.
- Backend is the only place that holds catalog/LLM/web-search credentials. The frontend calls it through an internal-secret-gated backendJson helper. This keeps secret surface area in one place and makes the system swap-friendly if we ever move the agent to a worker.
- Postgres + Redis are local Docker. Postgres holds auth, chats, messages, votes. Redis caches feed results, web search, and web-fetch payloads.

Process layout

Frontend (**frontend/**)

- Next.js 16 App Router, Turbopack dev, React Server Components.
- `app/(chat)/chat/[id]/page.tsx` — chat shell, loads message history via SWR.
- `app/page.tsx` — recommendation feed homepage.
- `components/chat/*` — message list, multimodal input, sidebar history, generative UI parts.
- `components/chat/shopping/*` — domain-specific UI: product-grid, product-card, comparison-table, seller-comparison, freshness-badge, generation-verdict-card, web-search-results, option-chips, buy-cta, assistant-response-json parser.
- `hooks/use-active-chat.tsx` — wraps `useChat` from `@ai-sdk/react` with transport that points at `${BASE}/api/chat`. Owns optimistic message state, SWR mutate on stream finish, and the auto-resume bridge.
- `lib/db/*` — Drizzle schema + queries (Postgres). Exposed only on the frontend Next.js server, which is allowed to read `POSTGRES_URL` for next-auth.
- `lib/feed/*` — typed client around the backend's `/api/feed/*` endpoints.
- `instrumentation.ts` — OTEL hooks (no-op in dev).

Backend (**backend/src/**)

- `server.ts` — single-file route handler using Hono primitives + Vercel AI SDK 6. Routes:

- `POST /api/chat` — opens an `createUIMessageStream` over a `streamText` call. Persists user message immediately; persists assistant message in `onFinish`.
- `GET /api/messages?chatId=...` — returns the chat row, message list (converted to UI message shape), and visibility.
- `POST /api/vote`, `GET /api/vote` — message votes.
- `GET /api/feed/recommendations`, `GET /api/feed/search` — non-chat catalog browse + search backed by Redis cache.
- `src/lib/agents/track1-shopping/` — the Track 1 agent definition.
- `src/lib/shopify/catalog.ts` — Shopify Catalog MCP JSON-RPC client. Normalizes UCP product and variant shapes into internal `CatalogProduct` / `CatalogVariant`. Sends `meta.ucp-agent.profile` per request. cache: "no-store" per Shopify catalog policy.
- `src/lib/ai/providers.ts` — model router. OpenRouter via `@openrouter/ai-sdk-provider`, DeepSeek via OpenAI-compatible base URL.
- `src/lib/db/queries.ts` — same Drizzle schema as the frontend, accessed via the back-end condition export.
- `src/lib/cache/redis.ts` — `getOrSet(key, ttl, fn)` helper for feed/web caches.

Storage

- Postgres 16 (Docker, port 5433 → 5432 inside).
- Redis 7 (Docker, port 6379).
- `docker-compose.yml` at the repo root brings both up with health-checks.

Agent design

Registry

`backend/src/lib/agents/registry.ts` maps `agentId` → `AgentDefinition { id, name, systemPrompt, tools, activeToolNames }`. Track 1 is live; tracks 2–5 are stubs with the same shape so they slot in later without surgery.

Track 1 tools (`backend/src/lib/agents/track1-shopping/tools/`)

Tool	Purpose	Backed by
<code>searchProducts</code>	Cross-merchant product search with <code>sortMode</code> , <code>freshnessHint</code> , <code>lightShuffle</code>	Shopify Catalog MCP <code>search_catalog</code>
<code>refineSearch</code>	Merges new keywords into prior query within the same chat	Same MCP, with <code>SearchMemo</code>
<code>showMore</code>	Paginates prior query, filters seen IDs, seeded shuffle within unseen	Same MCP
<code>getProduct</code>	Single-product detail	MCP <code>get_product</code> with <code>lookup_catalog</code> fallback
<code>compareProducts</code>	2–4 products side-by-side	Multiple <code>lookupProduct</code> calls

Tool	Purpose	Backed by
<code>compareSellers</code>	All variants/merchants for one product	Variant data already in catalog payload
<code>displayProducts</code>	Carousel rendering trigger (UI surface)	Memo of last results
<code>clarifyIntent</code>	Quick-reply chip menu	UI tool, no external call
<code>assessProductFreshness</code>	Wraps the freshness lib	Curated chipset + signals tables
<code>webSearch</code>	Bing → DuckDuckGo → Google fallback	Playwright + stealth + optional proxy
<code>webFetch</code>	Single-URL content fetch	Playwright
<code>buyProduct</code>	Emits checkout CTA card	UI tool, pass-through

Search memo

A per-request `SearchMemo` (last query, filters, result IDs, products, `updatedThisTurn` flag) is built from the conversation by `hydrateSearchMemo()` on every chat call, passed into the tool factories via `buildAgentForChat({ chatId, initialSearchMemo })`, and read/written by `searchProducts`, `refineSearch`, `showMore`, `displayProducts`. No DB persistence — it is reconstructed from message history on every request.

Per-chat variation

Three deterministic but per-chat-varied knobs all keyed off `chatId`: 1. `agentSeed` (32-bit hash) injected into the system prompt with a personality hint picked from a pool of 8. 2. `searchProducts` shuffle — `lightShuffle` keeps top-2 stable, seeds the long tail with `chatSeed ^ hash(query)` for relevance mode. 3. `suggestedActions` and `greeting` pick from rotation pools keyed by `chatId`.

Same chat reload → same outputs. Different chat → fresh phrasing, same facts.

Stop conditions

`stopWhen: [stepCountIs(10), hasToolCall("clarifyIntent")]`. Clarification is treated as terminal so the model cannot ask a question and then keep streaming product cards — that mode confused the UI and the buyer.

Generative UI contract

The frontend renders tool outputs, not model-emitted markdown. The contract is one-way: the model can only render a product card by successfully calling a tool that returns products. Specifically:

- `tool-displayProducts` → `<ProductGrid>` if `state === "output-available"` and products exist.
- `tool-compareProducts` → `<ComparisonTable>`. Only the last `compareProducts` call in a message renders, to dedupe multi-call drift.

- `tool-compareSellers` → `<SellerComparison>`.
- `tool-getProduct` → `<ProductDetailCard>` unless a `compareProducts` part is in the same message (then the card is suppressed because the table covers it).
- `tool-clarifyIntent` → `<OptionChips>`.
- `tool-buyProduct` → `<BuyCta>`.
- `tool-webSearch` → `<WebSearchResults>` (collapsed by default).
- `tool-getWeather`, `tool-createDocument`, `tool-updateDocument`, `tool-requestSuggestions` → vestigial from the chatbot template, unwired from the registry.

Defensive parsing of assistant text

Free-tier models occasionally violate the response-shape contract. Three guards in `components/chat/shopping/response-json.ts`:

1. `parseAssistantResponseText` strips a code fence, parses a top-level `{"response-Text": "..."} JSON` envelope, and returns the inner string. If the envelope is inline inside a larger text blob (model prepended prose), the parser finds the envelope substring, extracts the inner value, and joins prefix + value + suffix.
2. `stripMarkdownPipeTables` removes any markdown pipe table from assistant prose. The native `<ComparisonTable>` is the canonical comparison surface; a markdown copy would be duplicate noise.
3. `looksLikeClarifyMenuJson + tryParseClarifyMenuFromText` salvage cases where the model emits the `clarifyIntent` payload as text rather than calling the tool. The menu still renders as chips.

These guards mean the UI degrades gracefully when the model misbehaves, instead of leaking JSON or pipe tables into the chat.

Streaming and persistence

Stream

`/api/chat` returns an SSE stream produced by `createUIMessageStream` wrapping `stream-Text`. The Vercel AI SDK manages tool call/result framing. The backend writes a `data-chat-title` transient event when a title is generated for a new chat.

Persistence

- User message persisted before stream start (`server.ts:417`). Ensures the message is in the DB even if the stream dies.
- Assistant message persisted in `onFinish` (`server.ts:537`). Full parts array is saved including tool inputs, outputs, and the `data-thinking` summary part.
- On the frontend, `use-active-chat.tsx` `onFinish` awaits an SWR mutate of `/api/messages?chatId=.` and overwrites in-memory messages with the DB-fresh snapshot. This eliminates a class of bugs where a tool part stayed in `state="streaming"` on the client even though the backend had committed `state="output-available"`.

Auto-resume

`hooks/use-auto-resume.ts` calls `resumeStream()` if the most recent saved message is a user message — recovery for dropped streams. Replays via SSE; does not blindly re-run.

Caching strategy

Two-tier on the feed surface:

- Backend Redis via `getOrSet(key, ttl, fn)`:
 - `feed:recs:v1:<category>` — 1 hour TTL.
 - `feed:search:v1:<query>` — 5 minute TTL.
- Next.js Cache Components (`cacheComponents: true`) for the homepage shell prerender. Dynamic feed and search results stream inside `<Suspense>`.

Web search/fetch: - `webSearch` keyed on `(searchKind, num, query)`, in-memory TTL ~5 min.
- `webFetch` keyed on URL, in-memory TTL ~5 min.

Catalog search/lookup: not cached. Shopify's catalog policy prohibits caching search results and catalog images. All MCP calls use `cache: "no-store"`. Catalog images render directly from merchant URLs (no image proxy / optimizer).

Failure handling

Failure	Detection	Response
MCP returns 0 results or errors	Tool <code>execute()</code> result	Model is instructed to retry the same turn with broader/different terms (up to ~3 attempts) before telling the buyer.
MCP transient 503	Catch in <code>callCatalogTool</code>	Error thrown with status + body. Model surfaces a soft “couldn't reach catalog, retrying” path; usually resolves on next attempt.
LLM provider rate limit (429)	OpenRouter error	Provider router falls through to the next free model in the chain.
Stream drops mid-response	Frontend timeout (90s)	Inserts a synthetic “response timed out” assistant message + toast. User can retry from message actions.
Last DB message is user role on chat open	<code>use-auto-resume.ts</code>	Calls <code>resumeStream()</code> to recover from a dropped stream.
Model emits markdown pipe table	Frontend <code>stripMarkdownPipeTables</code>	Stripped before render.

Failure	Detection	Response
Model emits {"responseText":...} wrapper	Frontend parseAssistantResponseText	Unwrapped to inner string; surrounding prose kept.
Model emits clarify payload as text	Frontend tryParseClarifyMenuFromText	Renders chips anyway.
Tool output state stuck in "streaming" client-side	Frontend onFinish SWR mutate + setMessages	Replaced with DB-fresh snapshot when stream closes.
Web search blocked by Google CAPTCHA	Playwright + provider chain	Fallback to DuckDuckGo / Bing. Optional SEARCH_PROXY_URL for residential proxy.
Multiple compareProducts calls in one message	Frontend lastCompareProductsIndex	Only the last call's table renders.
Empty rating / specs in compare data	Frontend ComparisonTable	Entire row hidden if no product in the comparison has that field.
Low-rated product surfacing first	Backend filterLowRated in search-products.ts	Drops products with rating < 3.0 unless fewer than 3 survive the filter.
Stale product in fast-moving category	assessProductFreshness	Surfaces a generation verdict + suggests counterSearchHint.
Compatibility question without evidence	System prompt rule	Agent must run webSearch before answering; refuses if all retries fail.

Security and secrets

- All Shopify, OpenRouter, DeepSeek, web-search proxy, AUTH_SECRET, AI_GATEWAY, BLOB tokens live in .env files outside git.
- Backend exposes routes behind BACKEND_INTERNAL_SECRET — every frontend call carries this header. Catalog and feed read endpoints accept it without user auth (catalog data is not user-scoped).
- User auth via next-auth credentials provider, password hashed with bcrypt-ts. Guest mode is a real (anonymous) user row.
- No image proxying. Catalog images render directly from merchant CDN URLs per Shopify catalog policy.
- No catalog result caching. cache: "no-store" for every MCP request.

Limitations

- No multi-currency normalization in the comparison table — prices are shown in the variant's native currency. The agent will sometimes convert in prose, but the UI does not.
- Free-tier model artifacts. Markdown leaks, occasional JSON envelope leaks, and rare finishReason: "stop" after reasoning happen on the free model chain. All three are caught by frontend guards but the underlying model quality varies.

- No personalization. The agent does not consult prior chats. Per-chat seed is the only continuity mechanism.
- Catalog field coverage. Many Shopify merchants do not populate `topFeatures` / `techSpecs` / `rating`. The compare table now hides empty rows. We do not synthesize.
- No native mobile. Responsive web only.
- Tracks 2–5 are stubs. They share the agent shape but have no system prompts or tools yet.
- Web search depends on Playwright. Long-running, optional residential proxy may be needed on datacenter IPs.
- No streaming of titles to the sidebar. Title is generated in `onFinish` and shows on next list refresh.

How to extend

- New agent track: add an entry to `backend/src/lib/agents/registry.ts` with `id`, `name`, `systemPrompt`, `tools`, `activeToolNames`. Frontend `model/agent` selectors read from the same registry.
- New tool: add a file under `backend/src/lib/agents/track1-shopping/tools/`, export a `tool({...})`. Wire it into `agent.tools` and `agent.activeToolNames`. If it should render UI, add a `tool-<name>` branch in `frontend/components/chat/message.tsx`.
- New generative UI part: add a component under `frontend/components/chat/shopping/` and a corresponding render branch in `message.tsx`.
- Different catalog backend: replace `backend/src/lib/shopify/catalog.ts` with another implementation that returns the same `CatalogProduct` / `CatalogVariant` shape.